Maschinennahe Vernetzung Praktikum

von MQTT bis zu Node-Red

Contents

1 Tools 1
- 1.1 MQTT-Broker . . . 1
- 1.2 Interaktion mit dem Broker . . . 2
  - 1.2.1 Publisher . . . 2
  - 1.2.2 Subscriber . . . 2
  - 1.2.3 Broker . . . 2
  - 1.2.4 Topic . . . 3
  - 1.2.5 selbsterstellte Topics . . . 3
  - 1.2.6 Wildcards . . . 3
- 1.3 Modell-VPS . . . 4
  - 1.3.1 Modul 1 - Delivery . . . 4
  - 1.3.2 Modul 2 - Storage . . . 4
  - 1.3.3 Modul 3 - Dosing . . . 5
  - 1.3.4 Modul 4 - Production . . . 5
  - 1.3.5 Datenquelle: Sensoren . . . 5
  - 1.3.6 Datenquelle: Aktoren . . . 5
- 1.4 Datenformat JSON . . . 6

2 Node-Red 7
- 2.1 Message-Objekt msg . . . 7
- 2.2 Flow und Subflows . . . 7
- 2.3 flow basierte und globale Variablen . . . 7
- 2.4 Bausteine - Allgemein . . . 8
  - 2.4.1 inject . . . 8
  - 2.4.2 debug . . . 8
  - 2.4.3 comment . . . 8
- 2.5 Bausteine - Funktion . . . 8
  - 2.5.1 function . . . 8
  - 2.5.2 switch . . . 8
  - 2.5.3 change . . . 9
  - 2.5.4 delay . . . 9
  - 2.5.5 trigger . . . 9
- 2.6 Bausteine - Netzwerk . . . 9
  - 2.6.1 mqtt in . . . 10
  - 2.6.2 mqtt out . . . 10
- 2.7 Bausteine - Sequence . . . 10
  - 2.7.1 split . . . 10
  - 2.7.2 join . . . 11
- 2.8 Bausteine - Parser . . . 11

2.8.1 json 11

3 Konnektivität 11

4 Linkverzeichnis 11

1 Tools

1.1 MQTT-Broker

Für die Aufgabenstellung wird wichtig, dass man weiß, wo die Daten hingeschrieben und abgerufen werden können. Hierzu wird ein Broker benutzt, welches mittels MQTT-Protokolls Nachrichten empfangen und weiterleiten kann. MQTT steht für Message Queuing Telemetry Transport.

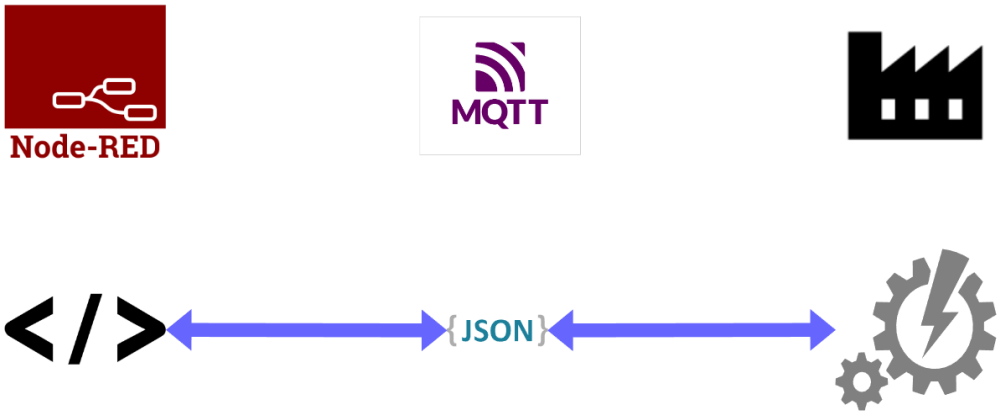


Diagram 1: Wo ist MQTT im Kontext einzuordnen?

MQTT ist hier als Kommunikationsschnittstelle zwischen Logikimplementation (Node-RED) und der Maschinensteuerung (Fabrikgebäude) vorgesehen. Die Daten werden in einem JSON-Format dargestellt.

1.2 Interaktion mit dem Broker

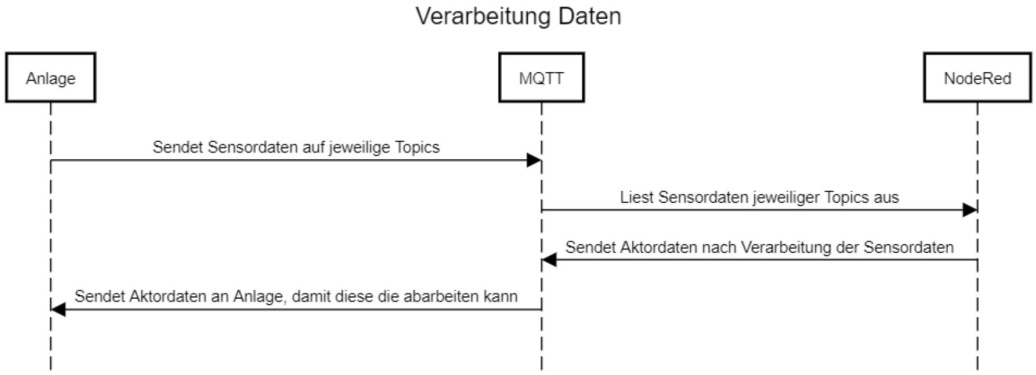


Diagram 2: Ablauf von Kommunikation

1.2.1 Publisher

Bei dem Vorgang des Publishens werden Daten auf den Broker geschrieben. Hier gibt der Publisher bei erfolgreicher Authentifikation vor, auf welches Topic welche Nachricht geschrieben werden soll. Diese Nachricht verteilt der Broker dann an die Subscriber.

1.2.2 Subscriber

Bei dem Vorgang des Subscribens werden Daten von dem Broker an den Subscriber abgerufen. Hier subscribed der Subscriber auf ein oder mehrere Topics. Sollten nun auf einem der Topics eine Nachricht eingehen, wird diese an jeden Subscriber auf dem Topic zugesendet.

1.2.3 Broker

Der Broker selbst ist wie ein Mittelmann: Er erhält Nachrichten von Publishern und leitet diese an die jeweiligen Subscriber weiter.Desweiteren merkt er, ob ein Subscriber oder Publisher sich verbindet oder die Verbindung trennt; ebenso wie das (Un-)Subscriben auf ein Topic.

1.2.4 Topic

Bei MQTT werden Nachrichten gerichtet weitergeleitet. Hierbei wird hierarchisch vorgegangen.

Ebene 0/Ebene 1/Ebene 2/...

Hierbei ist der Aufbau der Topics ähnlich der hierarchischen Struktur einer URL. Es ist wichtig, dass beim Subscriben der ganze Pfad angegeben wird, da nicht rekursiv geschaut werden kann.

1.2.5 selbsterstellte Topics

Natürlich kann man auch eigene Topics anlegen. Sofern ein Sub-Topic nicht existiert, wird dies mit der ersten Nachricht angelegt. Dementsprechend können unter dem Topic VPS beliebig viele Sub-Topics angelegt werden.

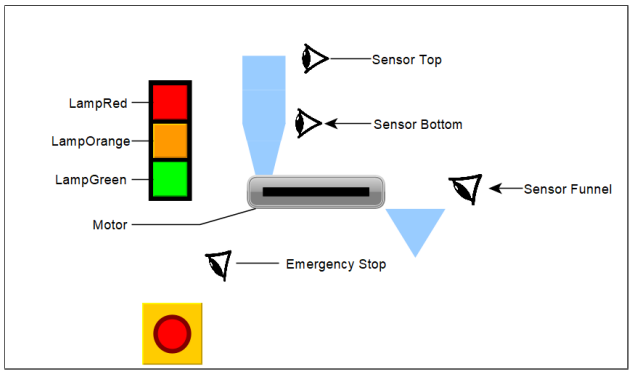
1.2.6 Wildcards

Hier unterscheidet man zwischen single und multi level wildcard.
Während bei dem single level die Position der Wildcard nur kontextuell wichtig ist, muss die multi level wildcard immer am Ende eines Pfades stehen.

1.3 Modell-VPS

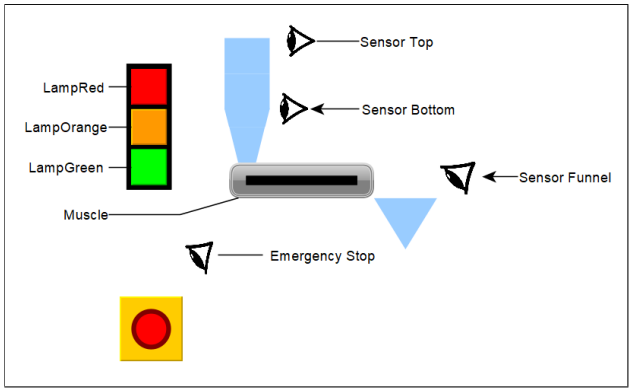
In folgenden Schemata/Diagrammen wird gezeigt, wie das jeweilige Modul aufgebaut ist und welche Sensoren wo liegen

1.3.1 Modul 1 - Delivery



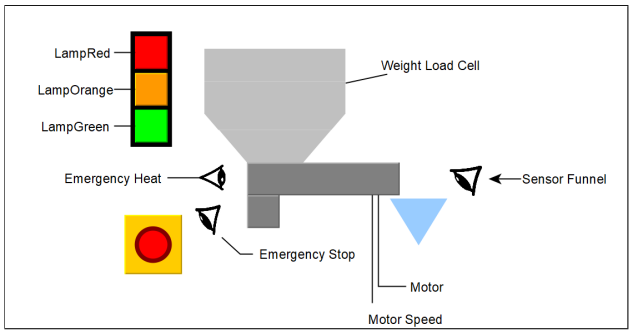
Modul 1 hat fünf Aktoren: 3 Lampen, 1 Motor und einmal Druckluft. Alle Aktoren werden per true und false gesteuert. Sobald der Motor ein true erhält geht auch die Luft automatisch mit an.

1.3.2 Modul 2 - Storage



Modul 2 hat fünf Aktoren: 3 Lampen, 1 pneumatischen Muskel und einmal Druckluft. Alle Aktoren werden per true und false gesteuert. Sobald der Muskel ein true erhält, geht auch die Luft automatisch mit an.

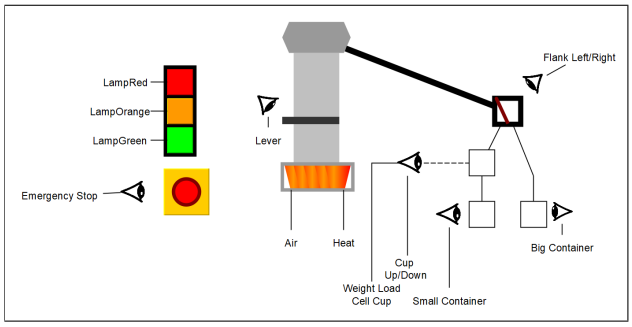
1.3.3 Modul 3 - Dosing



Modul 3 hat fünf Aktoren: 3 Lampen, 1 Transportschnecke und Druckluft. Fast alle Aktoren werden per true und false gesteuert. Sobald die Schnecke per einen Zahlwert bekommt und ein true erhält, bewegt sie sich in der Geschwindigkeit. Hierbei wird der Wert per PWM auf die Geschwindigkeit translatiert.

(Anstoß zum Nachlesen: Was bedeutet PWM und wie sind die Grenzwerte?)

1.3.4 Modul 4 - Production



Modul 4 hat sieben Aktoren: 3 Lampen, 1 Heizspirale, Druckluft sowie Becher und Flanke. Alle Aktoren an diesem Modul werden per true und false gesteuert.

1.3.5 Datenquelle: Sensoren

Hier unterscheidet man zwischen diskreten und kontinuierlichen Sensoren. Diskrete Sensoren liefern Wahrheitswerte zurück, beispielhaft Näherungssensoren. Kontinuierliche Sensoren liefern analoge Werte zurück, beispielhaft Wägezellen.

(Anstoß zum Nachlesen: Warum sind Signale von Sensoren standardmäßig true oder false? Was entscheidet das? Wofür steht ”normally closed” oder ”normally open”?)

1.3.6 Datenquelle: Aktoren

Ähnlich wie jedes Modul Sensoren hat, die ausgelesen werden können, so gibt es auch Aktoren, die beschrieben werden können. Hierbei sind es nicht nur logische Ausdrücke, sondern auch Zahlwerte, um beispielsweise Geschwindigkeiten angeben zu können.

1.4 Datenformat JSON

JSON (JavaScript Object Notation) ist ein leicht verständliches Format für den Datenaustausch. Es verwendet eine einfache Struktur von Schlüssel-Wert-Paaren, um Daten darzustellen, so dass sie für Menschen leicht zu lesen und zu schreiben und für Computer leicht zu analysieren und zu generieren sind. In den folgenden Abschnitten wird die JSON-Struktur von den verschiedenen Modulen beschrieben.

>w3schools hat da eine tolle Erklärung mit Ausprobieren!<

2 Node-Red

Node-Red ist ein online orientiertes Visual-Programming-Tool, basierend auf dem Node.js Framework. Per Drag and Drop werden hier Bausteine platziert und dann miteinander verbunden. Hierbei werden Daten in Form eines Message-Objekts zwischen den Nodes übergeben, verarbeitet und manipuliert. Die verschiedenen Nodes und Bausteine werden im Folgenden erklärt.

2.1 Message-Objekt msg

Die einfachste Art Nachrichten zwischen Nodes zu übermitteln, ist das Message-Objekt msg. Diese ist ein JavaScript Objekt, welches beliebig viele Eigenschaften haben kann. Eine Eigenschaft, die häufig verwendet wird, ist die Payload-Eigenschaft.
Wie das Message-Objekt genauer funktioniert kann hier nachgelesen werden:

>Node-RED Dokumentation: Messages<

2.2 Flow und Subflows

Innerhalb von Node-Red wird jeder Tab als Flow bezeichnet. Innerhalb eines Flows können ausgehend von Nodes Abläufe geplant und umgesetzt werden. Um Übersicht zu behalten, können Subflows genutzt werden.

2.3 flow basierte und globale Variablen

flow basierte Variablen sind Variablen, die im Flow überall zugänglich sind. globale Variablen funktionieren flowübergreifend.

2.4 Bausteine - Allgemein

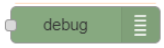
Im Reiter ”Allgemein” findet man Nodes, welche die ersten Schritte vereinfachen.

2.4.1 inject



Die inject node ermöglicht es, selbstbestimmte payloads zu generieren, sowie das Message-Objekt zu manipulieren. Man legt hier zu Beginn etwaige Felder innerhalb des Objekts fest.

2.4.2 debug



Die debug node gibt einen Output auf den Debug-Reiter im Node-Red-Interface.

2.4.3 comment

Mittels dieser node kann man Kommentare verfassen und somit Übersicht bei Linienführung schaffen.

2.5 Bausteine - Funktion

Diese Art von Nodes fokussiert sich auf funktionale Ausdrücke.

2.5.1 function

Die function node erlaubt es dem Nutzer Javascript Code auszuführen. Hierbei werden Referenzwerte, häufig von dem Message-Objekt msg, abgefragt, manipuliert oder weiterverarbeitet. Öffnet man die Node per Doppelklick, so kann über den Setup-Reiter die Ausgangsanzahl gesetzt werden.

2.5.2 switch

Die switch node ist eine Hilfe, um Abfragen zu gestalten. Ähnlich dem switch case aus Sprachen wie Java kann man verschiedene Vergleichsfälle erzeugen und somit verschiedene Ausgänge steuern lassen.

2.5.3 change

Die change node ermöglicht es, das Message-Objekt zu verändern, aber auch flow oder globale Variablen zu verändern.

2.5.4 delay

Die delay node verzögert das Weiterreichen des Message-Objekts um eine gewisse Zeit. Soll der Delay abgebrochen, schickt man eine Payload mit dem msg.reset Feld an die Node.

Warnung: Bei einem Reset geht die ursprüngliche Nachricht verloren!

2.5.5 trigger

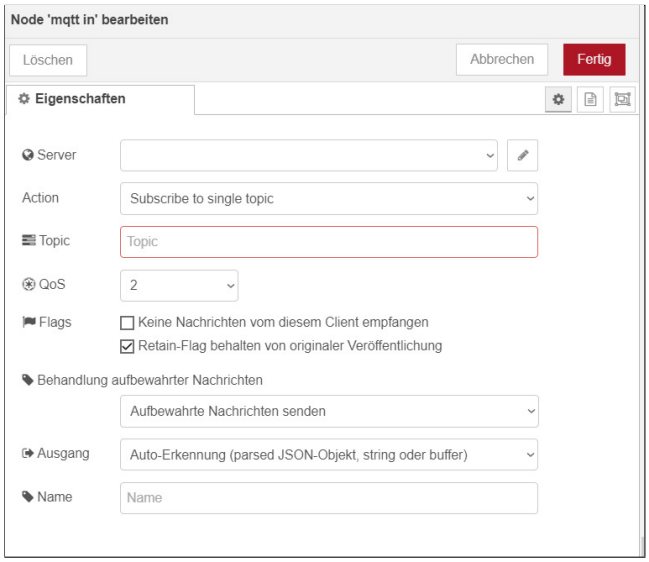
Ähnlich wie die delay node, kann die trigger node eine Nachricht verzögern. Hierbei kann bei Ansprechen der Node das Message-Objekt weitergeleitet werden, oder diese zurückgehalten werden, um dann als verzögertes zweites Message-Objekt gesendet zu werden.

Eine weitere Funktion dieser Node ist ein wiederholter Trigger. Diese Node ist genauso wie die delay node mittels msg.reset abbrechbar.

2.6 Bausteine - Netzwerk

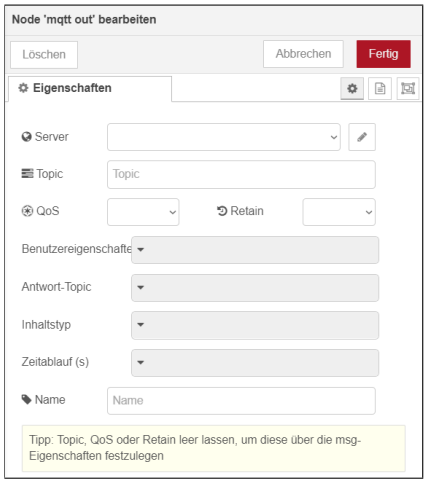
In diesem Abschnitt stellt Node-Red Netzwerkbausteine zur Verfügung, um Pakete über diese Netzdienste zu empfangen oder zu versenden. In dem Kontext des Praktikums wird hier nur die beiden mqtt I/O nodes behandelt und erklärt.

2.6.1 mqtt in



Hier werden die relevanten Daten eingetragen: Serveradresse, die jeweilige Aktion, Quality of Service (QoS) sowie das Topic auf welches subscribed werden soll.

2.6.2 mqtt out



Hier werden die relevanten Daten eingetragen: Serveradresse, Quality of Service (QoS) sowie das Topic auf welches gepublished werden soll.

2.7 Bausteine - Sequence

Nachrichten sind nicht immer nur exakt ein Wert. Häufig tauchen Sequenzen auf, welche mittels folgender Bausteine verarbeitet werden.

2.7.1 split

Die split node hat einen festgelegten delimiter, welcher die Payload des Message-Objektes auf delimiter untersucht und dann anhand dieser aufteilt. Diese Objekte werden nacheinander als einzelne Payloads versandt.

2.7.2 join

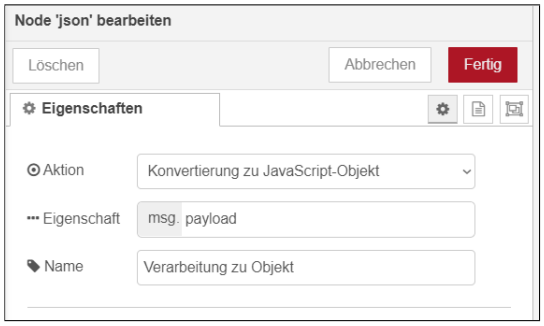
Das Gegenstück zu der split node ist die join node. Diese kann entweder automatisch zusammengehörige Payloads miteinander verknüpfen, oder einstellbar viele Einzelnachrichten.

Damit dies automatisiert funktioniert, muss das Message-Objekt ein Feld msg.parts haben, welches inkrementell nach oben zählen soll.
Sollte dies nicht der Fall sein, so muss überprüft werden, ob in der gejointen Nachricht beide Felder nicht die gleiche Payload haben, denn dann wurde nicht erfolgreich gejoined!

2.8 Bausteine - Parser

Parser erkennen Texte in gewissen Strukturen und können diese dementsprechend schnell vorverarbeiten oder in Objekte umwandeln. Da der Broker JSON Strings sendet, wird hier der JSON-Parser-Baustein erklärt.

2.8.1 json



Um eine Art Array zu erzeugen, nutzen wir die Funktion Konvertierung zu JavaScriptObjekt, da dieses aus dem String ein JSON-Array erzeugt. Die jeweilige value kann man gezielt über den jeweiligen key des key-value-pairs erreichen.

3 Konnektivität

Um Node-Red mit dem MQTT-Broker zu verbinden, benötigt man, wie oben genannt den MQTT Baustein. Egal ob in oder out, bei beiden Bausteinen bedarf es eines Brokers, dem man angeben sollte.
Die Daten für die Anbindung werden von der Lehrperson/Praktikumsbetreuung bekannt gegeben.

4 Linkverzeichnis

JSON = https://www.w3schools.com/js/js_json_objects.asp

Node-RED Msg = https://nodered.org/docs/user-guide/messages